Raft: understandable consensus

Raft is a consensus algorithm for managing a replicated log. A cluster of servers agrees on a single, ordered sequence of commands, so that every server's state machine applies the same commands in the same order and ends in the same state, even when some servers crash.

Roles

At any time each server is in one of three states: leader, follower, or candidate. In normal operation there is exactly one leader and all other servers are followers. Followers are passive: they issue no requests on their own but simply respond to requests from leaders and candidates. The leader handles all client requests. The candidate state is used to elect a new leader.

Terms

Raft divides time into terms of arbitrary length, numbered with consecutive integers. Each term begins with an election. If a candidate wins the election, it serves as leader for the rest of the term. Terms act as a logical clock: servers exchange their current term in every message, and a server that sees a larger term immediately updates its own term and reverts to follower. A server rejects any request carrying a stale term.

Leader election

Leaders send periodic heartbeats (AppendEntries messages that carry no log entries) to all followers to maintain their authority. If a follower receives no communication over a period called the election timeout, it assumes there is no viable leader and begins an election: it increments its current term, transitions to candidate, votes for itself, and sends RequestVote messages to every other server.

Each server votes for at most one candidate in a given term, on a first-come-first-served basis. A candidate wins if it receives votes from a majority of the servers in the full cluster. The majority rule ensures that at most one candidate can win the election for a particular term. Election timeouts are chosen randomly from a fixed interval, for example 150 to 300 milliseconds, which spreads out the servers so that in most cases only a single server times out, wins the election, and sends heartbeats before any other server times out. This randomisation makes split votes rare and resolves them quickly.

Log replication

Once a leader has been elected, it begins servicing client requests. Each request contains a command to be executed by the replicated state machines. The leader appends the command to its log as a new entry, then issues AppendEntries messages in parallel to each of the other servers to replicate the entry. Each log entry stores a command together with the term number when the entry was received by the leader, and an integer index identifying its position in the log.

When the entry has been safely replicated on a majority of servers, the leader considers it committed. The leader applies the entry to its state machine and returns the result to the client. Committed entries are durable and will eventually be executed by all of the available state machines. The leader includes the highest committed index in future AppendEntries messages so followers learn which entries they may apply.

Consistency check

When sending an AppendEntries message, the leader includes the index and term of the entry in its log that immediately precedes the new entries. If the follower does not find an entry in its log with the same index and term, it refuses the new entries. When a follower refuses, the leader decrements its nextIndex for that follower and retries, walking backwards until the logs match. The follower then deletes any conflicting entries and appends the leader's entries, so the follower's log becomes consistent with the leader's.

Safety

Raft guarantees that a leader for any given term contains all of the entries committed in previous terms. It enforces this during elections: a voter denies its vote if its own log is more up-to-date than the candidate's. Logs are compared by the term of the last entry, then by length. Because a committed entry is stored on a majority, and a candidate needs votes from a majority, at least one voter holds every committed entry and will refuse candidates that lack it.

Membership changes and snapshots

Clusters change membership through joint consensus, a transitional configuration in which decisions require majorities of both the old and the new configuration. To keep logs from growing without bound, each server independently takes snapshots of its state machine and discards the log entries the snapshot covers. A leader sends an InstallSnapshot message to followers that have fallen too far behind.